

Reconstruction of DOSY NMR signals - Part II

```
import os
import numpy as np
import matplotlib.pyplot as plt
```

Simulated Data

On importe une partie des fonctions utilisé par la Partie 1

```
class fun_minimize:
    # generic class for the minimisation problem
    def __init__(self,*args,**kwargs):
        return
    def __call__(self, *args, **kwds):
        # return f(x)
        return NotImplementedError
    def grad(self,
            *args, **kwds):
        # return gradient of f(x)
        return NotImplementedError
    def proximal(self,*args,**kwds):
        # return proximal operator of f(x)
        return NotImplementedError

def load_x(file_name:str):
    x = list()
    with open(file_name, "r") as f:
        for x_i in f.readlines():
            x.append(eval(x_i))
    return np.array(x)

import numpy as np
def get_X():
    X=np.loadtxt("x.txt")
    return X
X=get_X()
N=X.shape[0]

## Step 2: Create T using exponential sampling strategy
def construct_T(N=200):
    T_min=1
    T_max=1000
    T=np.zeros(N)
    for k in range(N):
        T[k]=T_min*np.exp(-(k)*np.log(T_min/T_max)/(N-1))
```

```

    return T
T=construct_T(N)

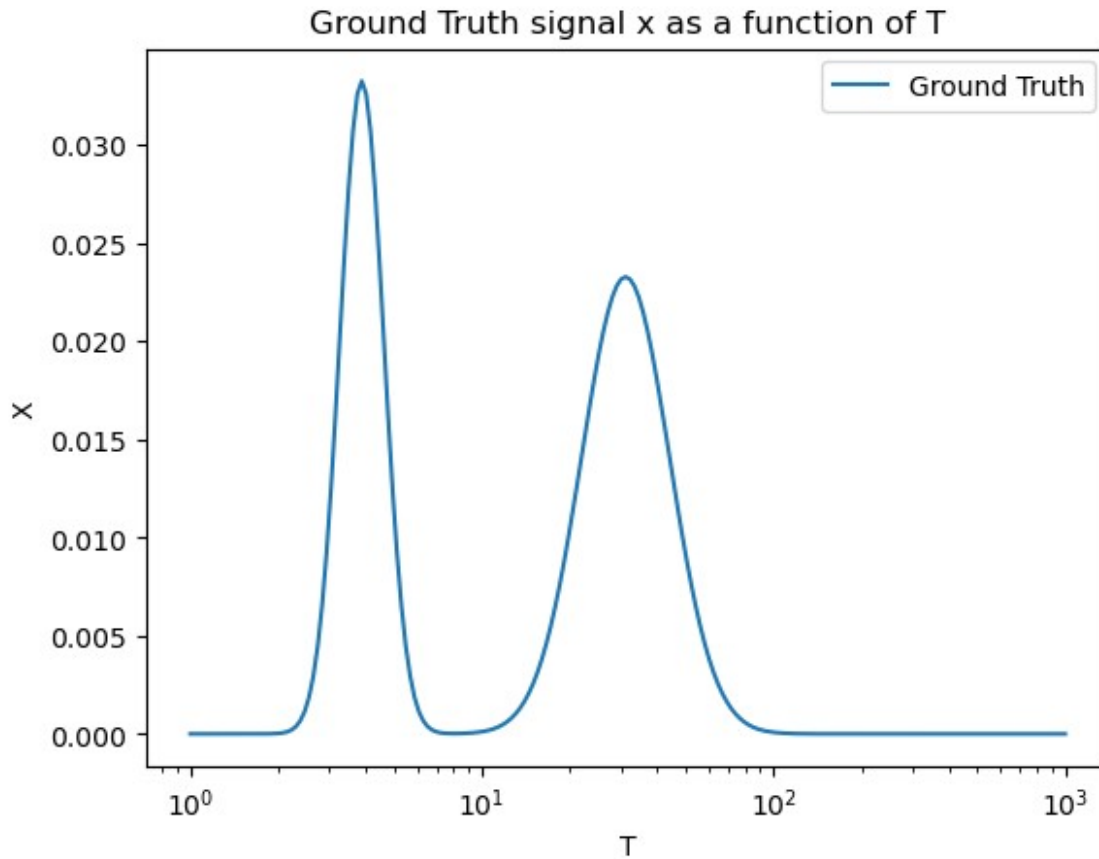
# Construct D
def construct_D(N):
    D=np.zeros((N,N))
    for n in range(N):
        D[n,n]=1
        D[n,(n-1)%N]=-1
    return D
D=construct_D(N)

import matplotlib.pyplot as plt
def plot_signal(T,X,title,label):
    plt.plot(T,X,label=label)
    plt.xlabel("T")
    plt.ylabel("X")
    plt.title(title)
    plt.legend()

title="Ground Truth signal x as a function of T"
label="Ground Truth"
plot_signal(T,X,title,label=label)
# add log scale on horizontal axis

plt.xscale('log')
plt.show()

```



```
def construct_t():
    t_min=0
    t_max=1.5
    M=50
    t=np.zeros(M)
    for m in range(M):
        t[m]=t_min+(m)/(M-1)*(t_max-t_min)
    return t
t=construct_t()
t.shape

(50,)

def construct_K(T,t):
    M=len(t)
    N=len(T)
    K=np.zeros((M,N))
    for m in range(M):
        for n in range(N):
            K[m,n]=np.exp(-T[n]*t[m])
    return K
K=construct_K(T,t)
```

```
def get_y():
    mu=0
    z=np.dot(K,X)
    sigma=0.01*z[0]
    w=np.random.normal(mu,sigma,z.shape[0])
    # simulate it
    y=np.dot(K,X)+w
    return y
y=get_y()
```

Q1: Is ent convex ? proper ? lower-semicontinuous ? differentiable ?

- **stricte convexité** : Comme $\text{ent}(x)$ est une somme de fonction $\varphi(x_i)$ séparables, il suffit de montrer que $\varphi(x)$ est strictement convexe pour montrer la stricte convexité totale. Montrons le, Sur le support de $\varphi(R_{\epsilon}^{+\epsilon,\epsilon})$, $u \log u$ est C^∞ sur $R_{\epsilon}^{+\epsilon,\epsilon}$ de dérivée seconde $1/u > 0$. Donc $u \log(u)$ est strictement convexe sur $R_{\epsilon}^{+\epsilon,\epsilon}$. Donc φ est strictement convexe, et ent par extension aussi.
- **propre**: φ est à valeurs finies sur $R_{\epsilon}^{+\epsilon,\epsilon}$, donc ent est à valeurs finies sur $\mathbb{R}$ et ent est donc propre.
- **semi-continue inférieurement**: Pour montrer que ent est semi-continue inférieurement, par séparabilité, il suffit de montrer que $\varphi(u)$ est semi-continue inférieurement. Or $\varphi(u) = \iota_{\epsilon,\epsilon}(u)$. Comme $\mathbb{R}$ est fermé convexe, la fonction ι est semi-continue inférieurement. $f(u)$ est continue donc semi-continue-inférieurement. Donc φ et ent sont semi-continue inférieurement.
- **différentiable** φ est non continue sur R et n'est donc pas différentiable.

Q2: Has the optimization problem a solution ? Is it unique ?

Comme montré au TP1, $\frac{1}{2} \|Kx - y\|^2$ est continue, positive, convexe et propre. De plus, on a

$\varphi(x) \xrightarrow{x \rightarrow +\infty} +\infty$ donc ent est coercive, convexe semi-continue inférieurement. donc

$\frac{1}{2} \|Kx - y\|^2 + \beta \text{ent}(x)$ est coercive, semi-continue inférieurement et propre, Ce problème admet donc une solution. Étant donné la stricte convexité de ent et la convexité de

$\frac{1}{2} \|Kx - y\|^2$, $\frac{1}{2} \|Kx - y\|^2 + \beta \text{ent}(x)$ est strictement convexe. il y alors unicité du minimum.

Q3: Give the expression of the proximity operator of ent at some $x \in \mathbb{R}^N$. (Hint : Use the Lambert W -function)

Calculons l'expression directement pour $\gamma > 0$. Comme ent est séparable, il suffit de trouver le proximal de φ . On a :

$$\text{prox}_{\gamma\varphi}(x) = \arg \min_{y \in \mathbb{R}} \gamma \log y + \frac{1}{2}(y - x)^2$$

Or $y \mapsto \gamma \log y + \frac{1}{2}(y - x)^2$ est dérivable sur $\mathbb{R}_{>0}$ de dérivée $\gamma \log y + \gamma + y - x$. On a alors :

$$\gamma \log y + \gamma + y - x = 0 \iff \gamma^{-1} y \exp(\gamma^{-1} y) = \gamma^{-1} \exp(\gamma^{-1} x - 1) \iff y = \gamma W(\gamma^{-1} \exp(\gamma^{-1} x - 1))$$

Comme la fonction φ est coercive et convexe, elle admet au moins un minimum. Ainsi, le point d'annulation de la dérivée de φ correspond à son unique minimum, appartenant à $\mathbb{R}_{>0}$ par positivité de W sur ce même ensemble. Ainsi, on a :

$$\text{prox}_{\gamma\text{ent}}(x) = \left(\gamma W(\gamma^{-1} \exp(\gamma^{-1} x_i - 1)) \right)_{1 \leq i \leq N}$$

Q4: Propose a forward-backward and a Douglas-Rachford algorithm to solve problem (1).

On étudie séparément le Forward Backward et l'algorithme de Douglas Rachford

Forward-Backward

$\frac{1}{2} \|Kx - y\|^2$ est C^∞ convexe et propre. Comme vu au TP précédent, elle est ν -lipschitz de gradient la plus grande valeur propre de $K^T K$. D'après 3), $\text{ent}(x)$ admet une expression proximale "simple" et est dans $\Gamma^0(\mathbb{R}^n)$. On est dans les bonnes hypothèses d'appliquer le forward backward.

Douglas-Rachford

Comme vu précédemment, $\frac{1}{2} \| Kx - y \|^2$ est dans $\Gamma^0(\mathbb{R}^n)$. Pour pouvoir appliquer Douglas Rachford, on doit, en plus des argument de Forward Backward, trouver une version proximale de $\frac{1}{2} \| Kx - y \|^2$. Comme c'est une fonction quadratique, l'expression de son proximal est

$$\text{prox}_{\gamma f}(x) = (id + \gamma K^T K)^{-1}(x + K^T y)$$

Q5: Implement them both and evaluate their performances for $\beta = 10^{-2}$.

Nous n'avons pas pu obtenir de résultats concluants pour $\beta=10^{-2}$, à la place nous montrons nos résultats à $\beta=10^{-3}$

```
# we gonna calculate v, the lipschitz gradient of g
A = K.T @ K + D.T @ D
v=np.linalg.eigvals(A)
print("the largest eigenvalue is ",v.max())
v_max=v.max()

the largest eigenvalue is  478.40084022768315

class f_square_norm(fun_minimize):
    def __init__(self,K,y,gamma=1):
        self.K=K
        self.y=y
        self.gamma=gamma
        self.inv_id_gamma_ll = np.linalg.inv(np.eye(K.shape[1]) +
self.gamma*K.T @ K)
        return
    def grad(self,x:"np.array")->"np.array":
        return self.K.T @ (self.K @ x -self.y )
    def proximal(self,x:"np.array")->"np.array":
        return self.inv_id_gamma_ll @ (x + self.gamma * K.T @ self.y)
    def __call__(self,x):
        return 1/2*np.linalg.norm(self.K @ x -self.y)**2

x_test=np.zeros(N)
f=f_square_norm(K,y)
print(f.grad(x_test).shape)

(200,)

from scipy.special import lambertw
```

```

def entropy(x):
    if np.any(x < 0):
        return np.inf

    phi = np.where(x > 0, x * np.log(np.maximum(x, 1e-10)), 0)

    return np.sum(phi)

class g_entropy(fun_minimize):
    def __init__(self, gamma):
        self.gamma = gamma
        return

    def proximal(self, x: "np.array") -> "np.array":
        prox = np.zeros_like(x)
        for i in range(len(x)):
            u = (x[i] / self.gamma) - 1 - np.log(self.gamma)
            if u > 100:
                prox[i] = self.gamma * (u - np.log(u))
            else:
                prox[i] = self.gamma * lambertw(np.exp(u)).real
        return prox

    def __call__(self, x):
        return entropy(x)

# random vectors
x_test = np.random.rand(N)
gamma_test = 0.1
g1_test = g_entropy(gamma_test)
print(g1_test.proximal(x_test).shape)
print(g1_test(x_test))

(200,)
-51.082140319272895

def forward_backward(x0, gamma, lambda_n, y, beta, num_iter, verbose=True):
    x_n = x0
    y_n = x0
    f = f_square_norm(K, y, gamma=gamma)
    g = g_entropy(gamma=gamma*beta)
    loss = []
    for n in range(num_iter):
        y_n = x_n - gamma*f.grad(x=x_n)
        x_n = x_n + lambda_n[n]*(g.proximal(x=y_n) - x_n)
        if n%10000==0 and verbose:
            print("iteration", n, "the loss is", f(x_n)+g(x_n))
        loss.append(f(x_n)+g(x_n))

    #print("iteration", n, "the loss is", f(x_n)+g(x_n))#"loss

```

```

f", f(x_n), "loss g", g(x_n))
    return x_n, loss, 2*g(x_n)

def douglas_rachford(x0, gamma, lambda_n, y, beta, num_iter, verbose=True):
    x_n=x0
    y_n=x0
    g=f_square_norm(K, y, gamma=gamma)
    f=g_entropy(gamma=gamma*beta)
    loss=[]
    for n in range(num_iter):
        y_n=g.proximal(x=x_n)
        z_n=f.proximal(x=2*y_n - x_n)
        x_n=x_n+lambda_n[n]*(z_n - y_n)
        if n%10000==0 and verbose:
            print("iteration", n, "the loss is", f(x_n)+g(x_n))
            loss.append(f(x_n)+g(x_n))

        #print("iteration", n, "the loss is", f(x_n)+g(x_n))#"loss
f", f(x_n), "loss g", g(x_n))
    return z_n, loss

# add hyperparameters for the backward forward method
num_iter=50000
# gamma must be between 0 and 2/v_max
gamma=1/v_max
# lambda must be between 0 nad delta= 2-(gamma*v_max/2)
delta=2-(gamma*v_max/2)
# lambda_n=[delta/2 for k in range(num_iter)]
lambda_n=[1 for _ in range(num_iter)]
beta = 1e-3

# init x0 as zeros
x0=np.zeros(N)

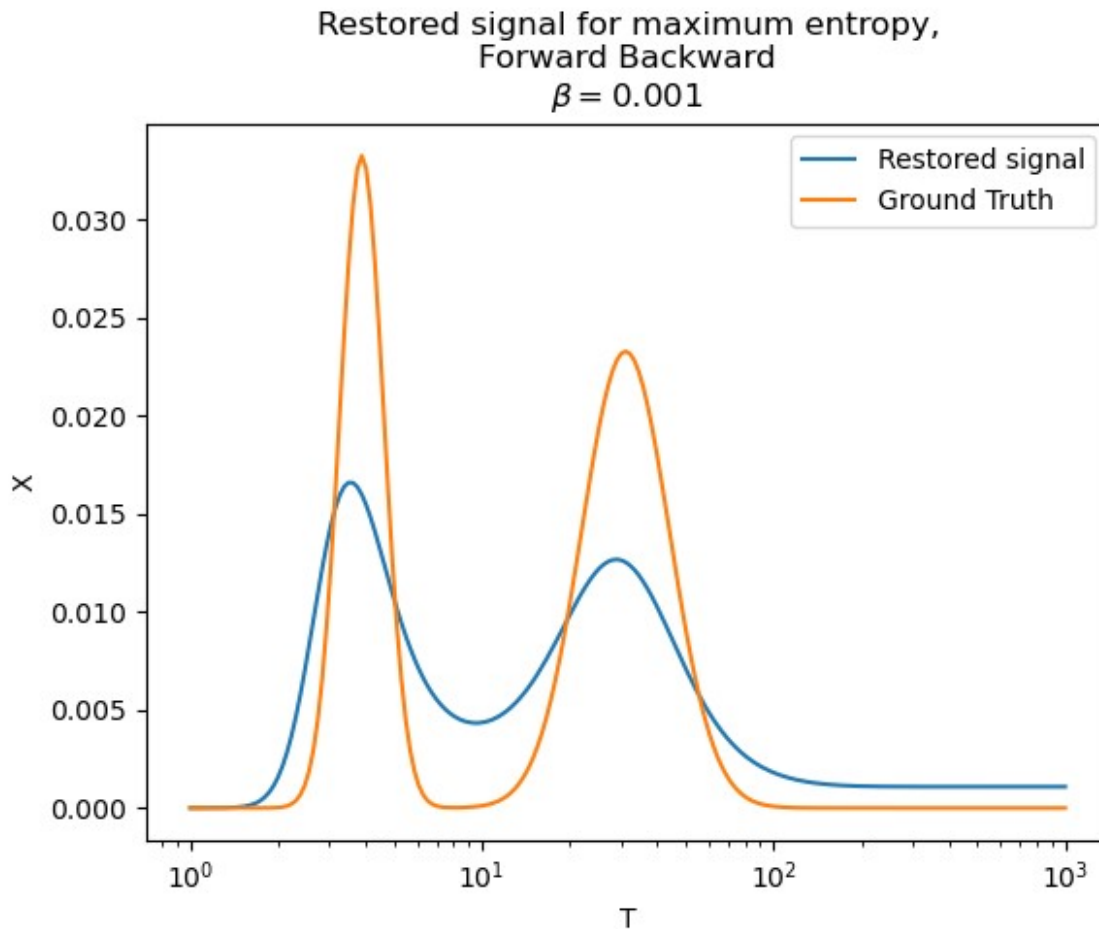
x_chap_entropy, loss,
_=forward_backward(x0=x0, gamma=gamma, lambda_n=lambda_n, y=y, beta=beta, num_iter=num_iter)
# plot the signal
title=f"Restored signal for maximum entropy, \nForward Backward\n${beta}"
label="Restored signal"
plot_signal(T, x_chap_entropy, title, label=label)
plot_signal(T, X, title, label="Ground Truth")
plt.xscale('log')

iteration 0 the loss is -4.038228076441357
iteration 10000 the loss is -4.840862927109661
iteration 20000 the loss is -4.838521176196343

```



```
iteration 30000 the loss is -4.838310432261047
iteration 40000 the loss is -4.838280026621303
```

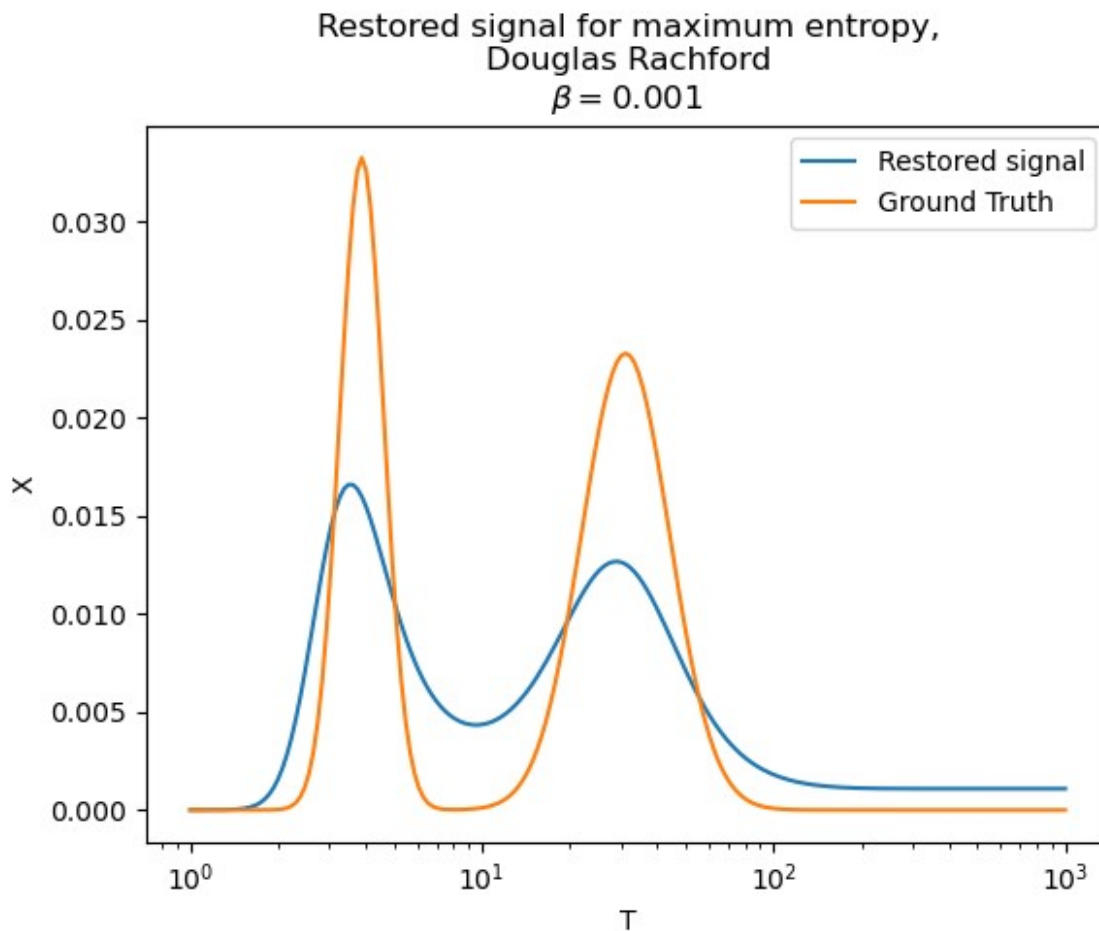


```
# init x0 randomly
x0= np.random.rand(N)

x_chap_entropy_dr,loss_dr=douglas_rachford(x0=np.zeros_like(x0),gamma=
gamma,lambda_n=lambda_n,y=y,beta=beta,num_iter=num_iter)
# plot the signal
title=f"Restored signal for maximum entropy,\nDouglas Rachford\n$\beta$\\
beta={beta}$"
label="Restored signal"
plot_signal(T,x_chap_entropy_dr,title,label=label)
plot_signal(T,X,title,label="Ground Truth")
plt.xscale('log')

iteration 0 the loss is -2.1751609720040985
iteration 10000 the loss is -4.853169604351958
iteration 20000 the loss is -4.850797804143775
```

```
iteration 30000 the loss is -4.850584091129561
iteration 40000 the loss is -4.85055323874511
```



Q6: What is the best choice for parameter β in terms of reconstruction error between the estimated object and the ground truth ?

```
from tqdm import tqdm

betas = np.logspace(-6, -2, 20)
errors_fb = []
errors_dr = []

for beta in tqdm(betas):
    x_chap_entropy_fb, loss,
    _=forward_backward(x0=x0,gamma=gamma,lambda_n=lambda_n,y=y,beta=beta,num_iter=num_iter,verbose=False)
```

```

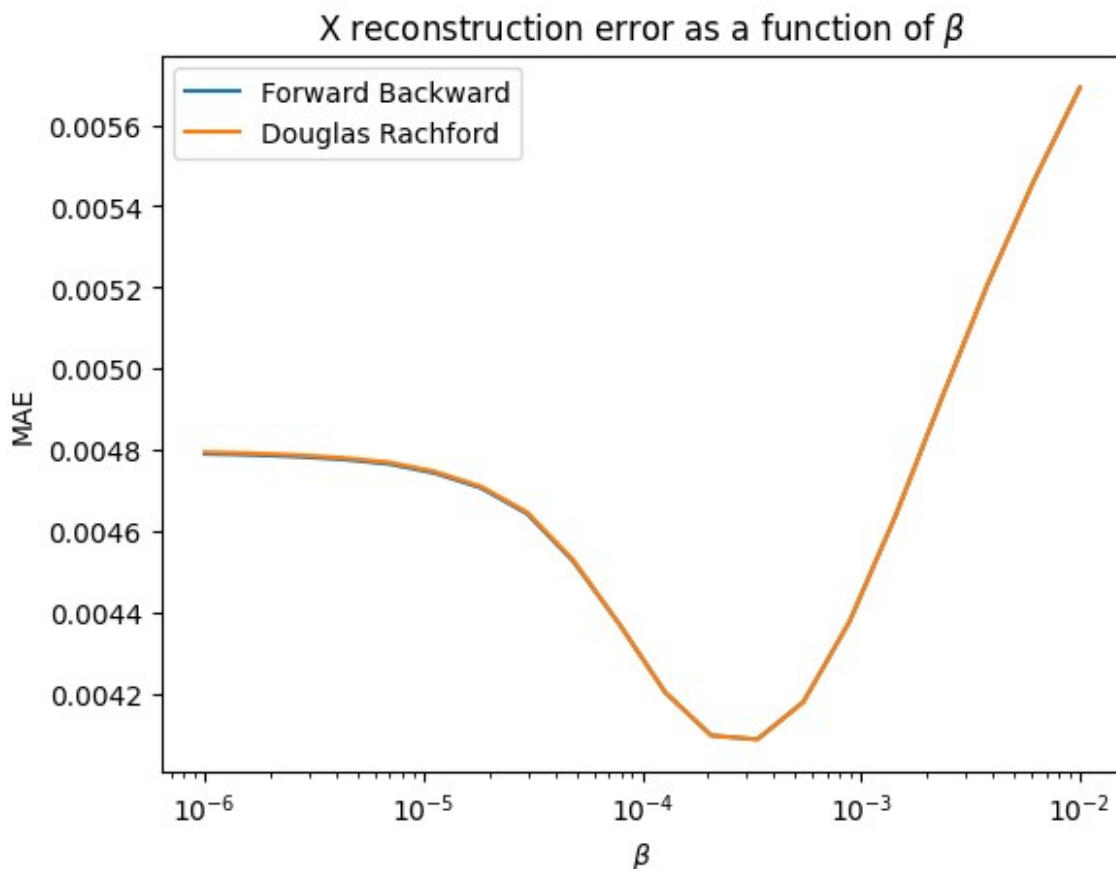
errors_fb.append(np.sum(np.abs(x_chap_entropy_fb -
X))/(len(x_chap_entropy)))

x_chap_entropy_dr, loss=douglas_rachford(x0=x0,gamma=gamma,lambda_n=lam
bda_n,y=y,beta=beta,num_iter=num_iter,verbose=False)
errors_dr.append(np.sum(np.abs(x_chap_entropy_dr -
X))/(len(x_chap_entropy)))

100%|██████████| 20/20 [07:59<00:00, 23.99s/it]

plt.plot(betas, errors_fb, label="Forward Backward")
plt.plot(betas, errors_dr, label="Douglas Rachford")
plt.xlabel("$\\beta$")
plt.ylabel("MAE")
plt.xscale("log")
plt.title("X reconstruction error as a function of $\\beta$")
plt.legend()
plt.show()

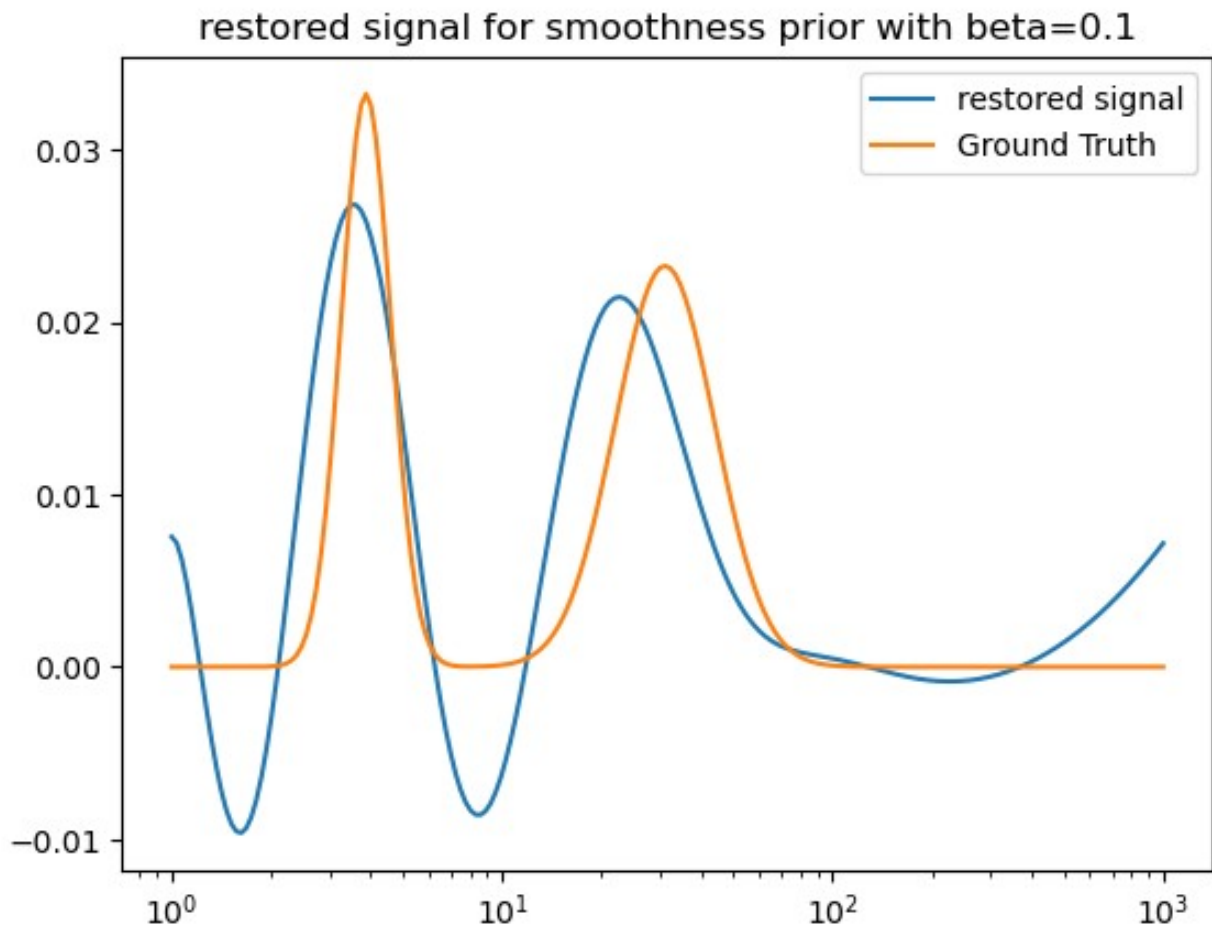
```



Le meilleur résultat est trouvé pour les deux algorithmes autour de $\beta = 2 * 10^{-3}$

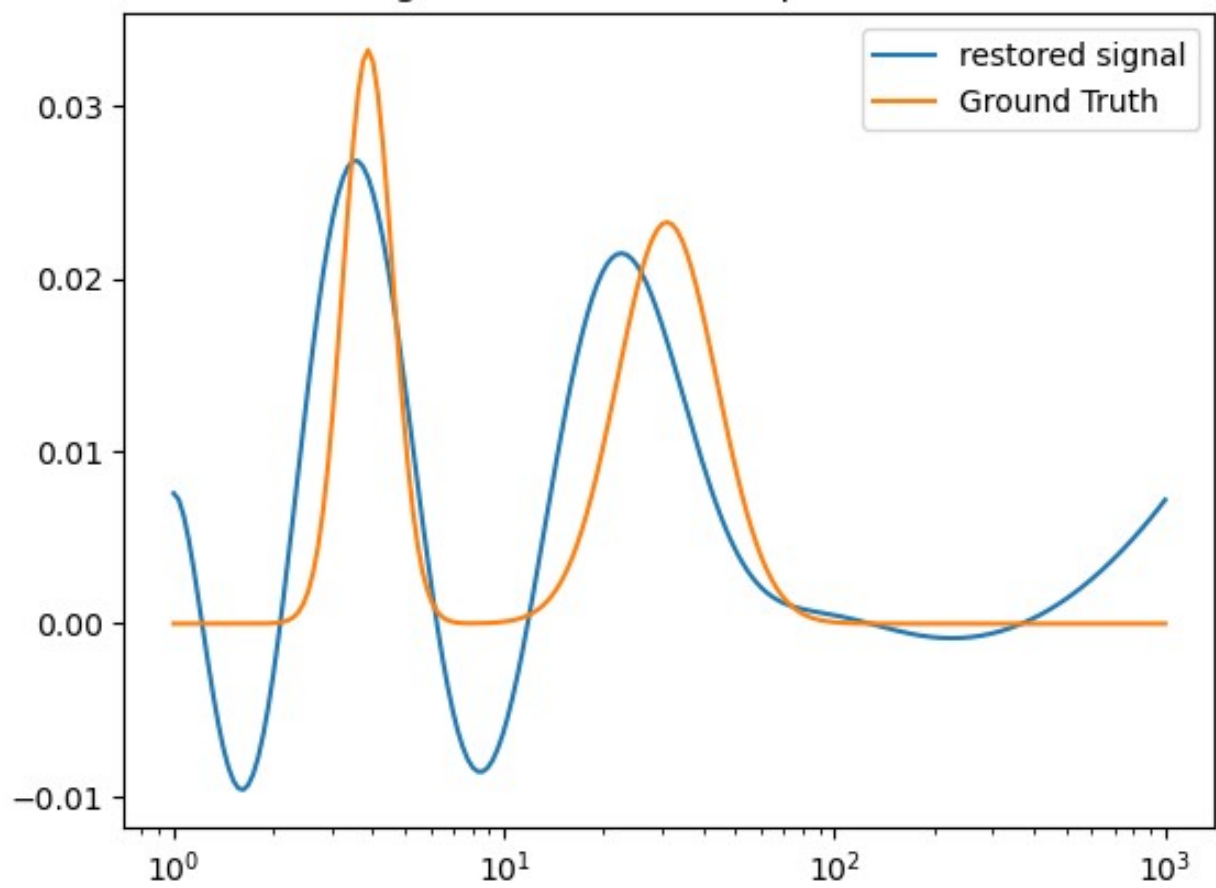
Q7 : Compare the maximum entropy regularization with the ones proposed in the previous part in terms of reconstruction quality.

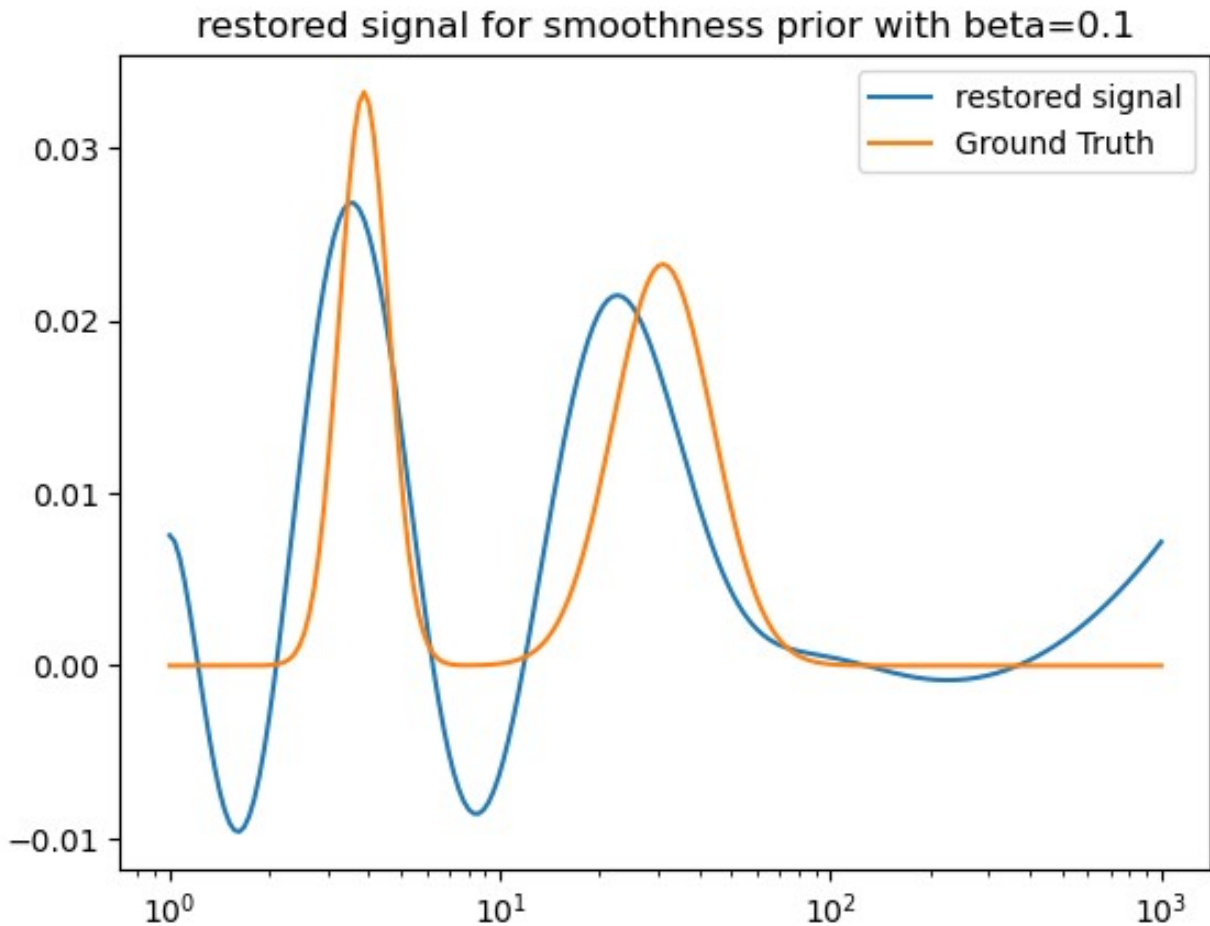
Résultat pour l'entropie



Résultat pour le sparsity prior du dernier TP

restored signal for smoothness prior with beta=0.1





Les deux méthodes de régularisation offrent des approximations différentes du signal réel en termes de qualité de reconstruction :

La **régularisation par entropie maximale** pénalise les variations abruptes du signal, ce qui conduit à une reconstruction plus lissée. Cependant, cette méthode introduit des écarts significatifs entre le signal reconstruit et la vérité de terrain, particulièrement dans les zones où le signal présente des pics prononcés. Cela résulte en une sous-estimation ou un lissage des valeurs des pics, comme illustré dans les graphiques.

En revanche, la **régularisation par "sparsity prior"** favorise une représentation parcimonieuse du signal, ce qui permet de mieux capturer les pics fins et localisés. Cette méthode est plus performante dans des contextes où le signal est composé d'événements rares et marqués, mais peut négliger certains détails ou transitions moins prononcées du signal.

Les résultats de l'entropie se rapprochent donc davantage de ceux obtenus avec un **smooth prior**, qui favorise une représentation localement cohérente du signal. La principale différence réside dans le fait que, avec l'entropie, le signal ne nécessite pas d'être périodique aux limites de l'intervalle considéré.

Q8: In practice, adjusting the parameter β may be difficult, while one has often ...

Avec cette nouvelle formulation, le problème d'optimisation équivaut à :

$$\arg \min_{x \in \mathbb{R}^N} \text{ent}(x) + \iota_{\{0, \eta M \sigma^2\}}(|Kx - y|)^2$$

$\Leftrightarrow$

$$\arg \min_{x \in \mathbb{R}^N} \text{ent}(x) + \iota_{B(y, \sqrt{\eta M \sigma^2})}(Kx)$$

où

$$\tilde{B} := B(y, \sqrt{\eta M \sigma^2})$$

est la boule centrée en y et de rayon $\sqrt{\eta M \sigma^2}$.

Nous sommes dans un cadre permettant d'utiliser l'algorithme d'optimisation Primal-Dual, qui résout

$$\text{Argmin } f + h + g \circ L$$

avec dans notre cas:

- $K \in \mathbb{R}^{N \times M}$ comme opérateur linéaire L ,
- $f := \text{ent} \in \Gamma_0(\mathbb{R}^N)$,
- $g := \iota_{\tilde{B}} \in \Gamma_0(\mathbb{R}^M)$,
- et $h=0$ (ayant un gradient β -Lipschitzien).

Nous avons déjà discuté de la manière de calculer $\text{prox}_{\tau f}$.

En utilisant la formule de décomposition de Moreau, nous obtenons :

$$\text{prox}_{\sigma g}(x) = x - \sigma \text{prox}_{\sigma^{-1}g}(\sigma^{-1}x).$$

Dans ce cas, puisque $g = \iota_{\tilde{B}}$, son prox est facile à calculer, car il s'agit simplement de la projection $P_{\tilde{B}}$, pour tout σ .

Pour projeter un vecteur x sur la boule centrée en y de rayon r , on peut simplement appliquer :

- $x \mapsto y + \frac{r}{\|x - y\|}(x - y)$ (le mappage sur la sphère) si x est en dehors de la boule,

- $x \mapsto x$ si x est à l'intérieur.

Primal Dual Implementation

```
class g_indicator_ball(fun_minimize):
    def __init__(self, y, radius, sigma):
        self.y = y
        self.radius = radius
        self.sigma = sigma

    def proximal(self, x: "np.array") -> "np.array":
        """Projection onto the ball"""
        diff = x - self.y
        norm_diff = np.linalg.norm(diff)

        if norm_diff <= self.radius:
            return x
        else:
            return self.y + (self.radius / norm_diff) * diff

    def proximal_conjugate(self, v: "np.array") -> "np.array":
        """Moreau decomposition formula"""
        return v - self.sigma*self.proximal(v/self.sigma)

    def __call__(self, x):
        if np.linalg.norm(x - self.y) <= self.radius:
            return 0
        else:
            return np.inf

def primal_dual(x0,v0,tau,sigma,y,K,radius,num_iter,verbose=True):
    x_n=x0
    v_n=v0
    f=g_entropy(gamma=tau)
    g=g_indicator_ball(y, radius, sigma=sigma)
    square_norm=f_square_norm(K,y)
    loss=[]
    for n in range(num_iter):
        x_n_1=f.proximal(x_n - tau*K.T @ v_n)
        v_n=g.proximal_conjugate(v_n + sigma*K@(2*x_n_1 - x_n))
        x_n=x_n_1
        if n%10000==0 and verbose:
            print("iteration",n,"the loss is",f(x_n)+square_norm(x_n))
        loss.append(f(x_n)+square_norm(x_n))

        #print("iteration",n,"the loss is",f(x_n)+g(x_n))#"loss
    f",f(x_n),"loss g",g(x_n))
    return x_n,loss

num_iter=50000
sgm=.5
```



```

op_norm_K = np.linalg.norm(K, ord=2)
tau = 1.0 / (sgm * op_norm_K**2 + 1) # Ensure  $\tau-1 \geq |\sigma||K||^2$ 
eta=.2
sigma=0.01
M=len(x0)
radius=np.sqrt(eta*M*sigma**2)
x_chap_entropy_pd,loss_pd=primal_dual(x0=np.zeros_like(x0),v0=np.zeros_like(y),tau=tau,sigma=sgm,y=y,K=K,radius=radius,num_iter=num_iter)

iteration 0 the loss is -9.271787448087562
iteration 10000 the loss is -4.7562976271438435
iteration 20000 the loss is -4.625361819710193
iteration 30000 the loss is -4.544048683716919
iteration 40000 the loss is -4.484561127025125

# plot the signal
title=f"Restored signal for maximum entropy,\nPrimal Dual method"
label="Restored signal"
plot_signal(T,x_chap_entropy_pd,title,label=label)
plot_signal(T,X,title,label="Ground Truth")
plt.xscale('log')

```

